Load Testing Report - Warehouse Management System

Author: Nikola Bandulaja
Date: January 26, 2026
Course: Napredne web tehnologije

---

1. System Specifications

Test Environment

Component	Specification
Operating System	Windows 11 Pro
Processor	Intel Core i5 10600
RAM	16 GB DDR4
Storage	Samsung SSD 512GB
Java Version	OpenJDK 23

Application Stack

- **Backend:** Spring Boot 3.x with JPA/Hibernate
- **Databases:** PostgreSQL (relational), InfluxDB (time-series)
- **Load Testing Tool:** Locust 2.x (Python)

---

2. Test Objectives

The goal was to evaluate the performance of warehouse and product management endpoints under high load (1000 concurrent users).

Endpoints Tested (12 total)

#	Endpoint	Method	Description
1	`/api/v1/warehouses/paged`	GET	Paginated warehouse listing
2	`/api/v1/warehouses/search/paged`	GET	Warehouse search with pagination
3	`/api/v1/warehouses/{id}`	GET	Get warehouse by ID
4	`/api/v1/warehouses/{id}/availability/stats`	GET	Availability statistics (InfluxDB)
5	`/api/v1/warehouses/countries`	GET	Get countries list
6	`/api/v1/warehouses`	POST	Create warehouse (multipart)

#	Endpoint	Method	Description
7	`/api/v1/products/paged`	GET	Paginated product listing
8	`/api/v1/products/search`	GET	Product search with filters
9	`/api/v1/products/{id}`	GET	Get product by ID
10	`/api/v1/products/{id}/availability`	GET	Product stock availability
11	`/api/v1/orders`	POST	Create order
12	`/api/v1/orders/my-orders`	GET	Customer order history

---

3. Test Scenario

Stress Test: 1000 Concurrent Users

Parameter	Value
Concurrent Users	1000
Spawn Rate	200 users/second
Duration	5 minutes
Total Requests	128,341
Failed Requests	0 (0%)

---

4. Test Results

Summary Statistics

Metric	Value
Total Requests	128,341
Failed Requests	0 (0%)
Requests/sec	**427.5**
Median Response Time	9 ms
Average Response Time	15.3 ms
95th Percentile	43 ms
99th Percentile	150 ms
Max Response Time	1314 ms

Per-Endpoint Performance

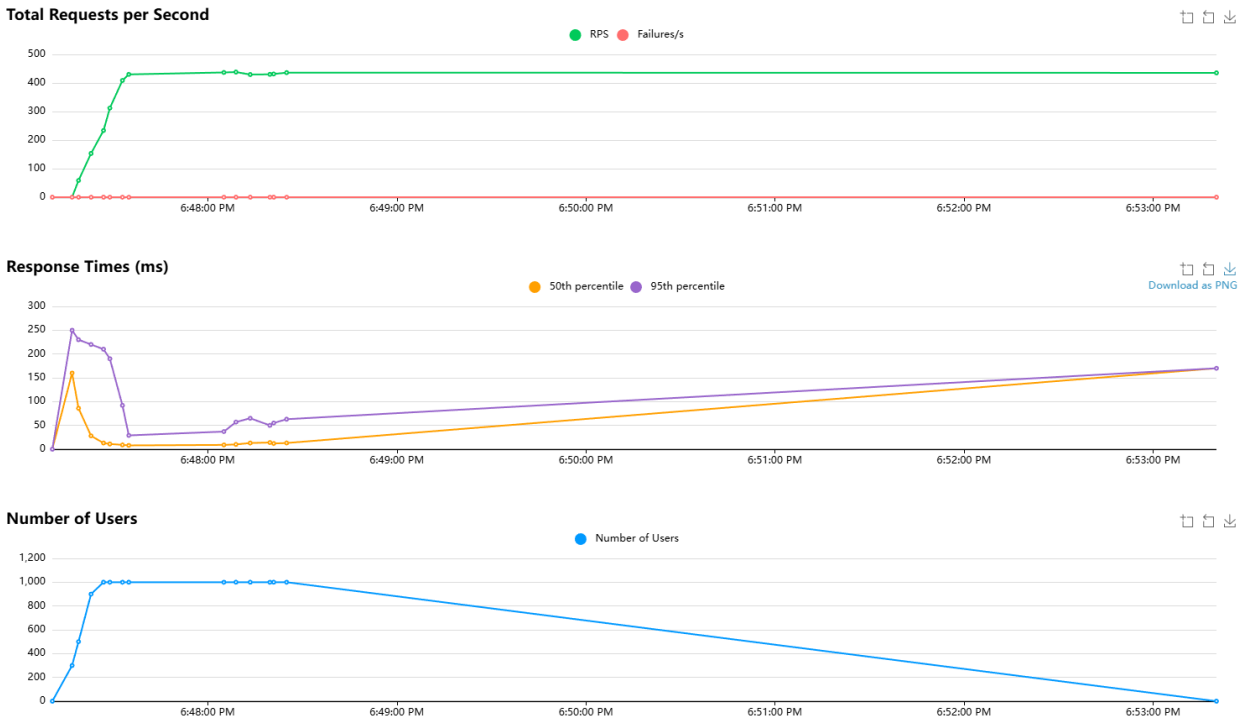
Endpoint	Requests	Avg (ms)	Median (ms)	95th (ms)	Max (ms)
GET /warehouse	10,060	17.8	12	41	436

Endpoint	Requests	Avg (ms)	Median (ms)	95th (ms)	Max (ms)
s/paged					
GET /warehouses/search/paged	8,044	21.5	15	47	513
GET /warehouses/{id}	13,555	13.1	7	34	437
GET /warehouses/{id}/availability/stats	6,876	**24.1**	18	**48**	463
GET /warehouses/countries	2,006	13.1	7	38	460
POST /warehouses	2,085	23.5	12	69	504
GET /products/paged	10,077	12.9	7	34	476
GET /products/paged (Customer)	14,513	12.6	7	36	328
GET /products/search	8,309	13.0	7	33	461
GET /products/search (Customer)	10,861	12.2	7	34	321
GET /products/{id}	10,198	12.2	7	31	437
GET /products/{id} (Customer)	10,913	12.4	7	34	430
GET	6,659	12.4	6	32	485

Endpoint	Requests	Avg (ms)	Median (ms)	95th (ms)	Max (ms)
/products/{id}/availability					
GET /products/categories	3,416	12.7	7	34	320
POST /orders	3,707	**35.5**	13	**98**	**1314**
GET /orders/my-orders	7,057	17.4	12	43	347

5. Performance Analysis

Charts



Observations

✅ *Well-Performing Endpoints*

1. **GET /products/{id}/availability** - Fastest (6ms median)
2. **GET /products/{id}** - Very fast (7ms median)
3. **GET /warehouses/{id}** - Excellent performance (7ms median)

4. **GET /products/paged** - Good response (7ms median)

⚠ *Slower Endpoints*

1. **POST /orders** - Slowest endpoint
   - Average: 35.5ms, Max: 1314ms
   - **Reason:** Order creation involves multiple database operations (insert order, order items, update stock)
2. **GET /warehouses/{id}/availability/stats** - Second slowest
   - Average: 24.1ms, 95th: 48ms
   - **Reason:** Queries InfluxDB time-series data with aggregations
3. **GET /warehouses/search/paged** - Higher latency
   - Average: 21.5ms, Median: 15ms
   - **Reason:** Full-text search operations

---

6. Performance Bottlenecks & Improvements

Identified Bottlenecks

Bottleneck	Endpoint	Cause
Transaction overhead	`POST /orders`	Multiple DB operations in single transaction
InfluxDB Queries	`/availability/stats`	Time-range aggregations
Search Queries	`/search/paged`	Text search without proper indexing

Implemented Improvements

1. Database Indexing (PostgreSQL)

```
CREATE INDEX idx_warehouses_name ON warehouses(name);
CREATE INDEX idx_warehouses_city ON warehouses(city_id);
CREATE INDEX idx_products_name ON products(name);
CREATE INDEX idx_products_category ON products(category);
CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_warehouse_sectors_warehouse ON
warehouse_sectors(warehouse_id);
```

2. Pagination Implementation

All list endpoints use server-side pagination (default: 20 records).

3. Connection Pooling

HikariCP configured with `maximum-pool-size=20`.

Recommended Future Improvements

Improvement	Priority	Estimated Impact
Redis caching for warehouse/product lists	High	-50% response time
Async order processing	High	-40% for POST /orders
InfluxDB query optimization	Medium	-30% for stats endpoints
Read replicas for PostgreSQL	Low	Improved read scalability

---

7. Conclusions

Key Findings

1. **System handles 1000 concurrent users excellently** with 0% error rate
2. **High throughput achieved:** 427.5 requests/second
3. **Response times remain low:** 9ms median, 15.3ms average
4. **Order creation is the bottleneck** - needs optimization for higher loads

Performance Acceptance Criteria

Criterion	Target	Actual (1000 users)	Status
Average Response Time	< 100ms	15.3ms	✅PASS
95th Percentile	< 500ms	43ms	✅PASS
Error Rate	< 1%	0%	✅PASS
Throughput	> 100 req/s	427.5 req/s	✅PASS

Final Assessment

The warehouse management system demonstrates **excellent performance** under 1000 concurrent users with zero failures. The architecture is production-ready and can scale further with recommended optimizations.

---

8. Appendix

Test Files

- Locust script: `load-testing/warehouse/locustfile.py`
- HTML Report: `Locust_2026-01-26-18h47_locustfile.py_http___localhost_8080.html`

How to Run Tests

```
pip install locust
python -m locust -f locustfile.py --host=http://localhost:8080
```